

Originally published in this reference documentation issue, and then copied into Runbooks.

Quick reference

Cause

Severe **BufferMapping** contention indicates postgres's shared buffers pool is thrashing:

- This flavor of LWLock contention emerges from churn on the shared buffer pool.
- The workload's working set no longer fits well into the pool of shared buffers.
- Evictions occur frequently enough to severely degrade queries.
- The instance-wide bottleneck becomes contention for the **BufferMapping** LWLocks, which mediate evicting pages from shared buffers.
- This can only happen when the working set size does not fit in the pool of shared buffers but still fits in the filesystem cache. If the filesystem cache also had a high miss rate, physical disk IO latency would become the bottleneck, rather than the shared buffer pool's **BufferMapping** lock acquisition rate.

Mitigation first response

Reduce query concurrency per postgres instance:

- If affecting replicas, add another replica.
- If affecting primary or if cannot add replicas quickly, moderately reduce the appropriate pgbouncers' backend connection pool max size.
- Reducing per-instance query concurrency should reduce working set size and shared buffer churn. This reduces eviction overhead, improving query response time and possibly also query throughput.

Analysis

- In prometheus metrics, what activity patterns recently changed? Increases in fetched rows per second may hint at what is driving churn on the shared buffer pool (which leads to contention beyond a certain churn rate).
 - rate of index scans on each table or index
 - ratio of index fetches per index scan on each index of any large or interesting table
- In snapshots of `pg_stat_activity`, were any queries showing unexpectedly high concurrency? High frequency queries that touch many shared buffers may significantly contribute to buffer churn.
- In the `auto_explain` log events in `postgresql.csv` logs, what queries show up often?
 - Which part of their execution plan touch the most shared buffers?

- This may help identify a specific index or table to focus on. A moderate buffer usage on a frequently run query can still drive the working set size far enough above the pool size to cause severe contention.

Background Details

Symptoms

When we have an incident that emerge from **BufferMapping** lightweight lock contention, usually the cause is not obvious.

The more immediately visible symptoms tend to be widespread slowness and timeout errors affecting a wide variety of customer-facing endpoints – a generic “site degraded or down” suite of symptoms. Those symptoms emerge from the saturation of a resource in a common critical path shared by many endpoints (in this case one or more postgres instances).

Once we determine the bottleneck to be somewhere in the database, there are several ways to discover if **BufferMapping** LWLock contention is consuming a significant amount of time: the `pg_stat_activity` view, the `pg_wait_sampling` extension, or the prometheus metrics or elasticsearch log events based on sampling them.

So what does **BufferMapping** contention mean?

Essentially the shared buffer pool is thrashing.

The shared buffer pool is a fixed-sized pool of 8 KB buffers, each holding a single 8 KB page of data from a file backing a specific table or index. (Custom builds of postgres may use other page sizes, but 8 KB pages are the default.)

Each time a running query needs to access table/index data from a file, it must read that page of file data into a shared buffer. If that particular page happens to already be in an existing shared buffer, it can use that buffer as-is. Otherwise, it has to choose a shared buffer and read that page into it. Because the shared buffer pool is a fixed size, reading a page into a buffer almost always requires evicting some other page from a buffer. This eviction is handled by the **BufferAlloc** function (source). In the usual case where eviction is required, it must briefly obtain an exclusive-mode lock on a specific 1 or 2 of the 16 **BufferMapping** lightweight locks (source). Obtaining those locks guarantees concurrency safety when updating the accounting metadata for the buffer – disassociating it with its old page contents and reassociating it with its new page contents.

Contention dynamics

Here’s where concurrency comes into play.

Obtaining those 1 or 2 of those 16 locks in exclusive-mode conflicts with any other backends concurrently holding either lock in share-mode. There can be many concurrent share-mode lock holders; those do not conflict with each other. But whenever a backend needs exclusive-mode, it must both wait for existing lock holders and block subsequent lock attempts. In other words, each time a lock must be acquired in exclusive-mode, it acts as a serialization point, impeding all of the subsequent lockers for that portion of the shared buffer pool, even when they only want share-mode.

This effectively reduces query concurrency from N backends down to somewhere between 16 and 1 (trending towards 1 as the contention severity increases). This concurrency reduction lasts for whatever brief timespan exclusive-mode is in effect. Lightweight locks are typically held very briefly – often just a few microseconds between acquire and release. But when exclusive-mode is acquired too often, that effectively reduces the amount of time when multiple backends can concurrently make forward progress – reducing the system-wide query throughput and increasing query duration.

The impact of this serialization tends to be more severe when the normal number of concurrently active backends is larger, because:

- Each time exclusive-mode is needed, it blocks all other share-mode acquires of that lock.
- As more queries run concurrently, their collective working set size tends to grow, and that drives up eviction rate since the shared buffer pool's size is fixed.

So both the probability and severity of a performance regression tend to increase in tandem with:

- the shared buffer pool's eviction rate (i.e. the exclusive-mode lock acquisition rate of **BufferMapping** LWLocks)
- the number of concurrently active backends executing queries (since that increases working set size)
- the number of buffers visited by concurrently running queries
- a frequently run query whose execution plan touches thousands of buffers (e.g. such that its concurrency times buffer count sums to a non-trivial percentage of the shared buffer pool size **and** those concurrent executions are not likely to have a lot of overlap in the pages they access)

Managing eviction rate: pool size, concurrency, execution plans for frequent queries

For **BufferMapping** LWLocks, exclusive mode is used for shared buffer eviction (as well as less common cases like invalidating a buffer when its backing page is being deleted).

Evictions are normal and happen regularly in virtually all dbs. Most contention events are trivially short, resolving in microseconds. This only becomes signifi-

cant when the eviction rate causes enough contention to affect query response times and impede the query throughput rate.

When the eviction rate increases to the point where we can observe **BufferMapping** lock wait events affecting many backends concurrently, that's a clear sign that the shared buffer pool is not big enough to hold the current "working set" of pages that are collectively required to satisfy the concurrently running queries' execution plans. That is "cache thrashing" behavior: evicting pages frequently enough that collective overhead impedes throughput. A common flavor of this is when the working set size moderately exceeds the cache size; pages are repeatedly evicted and soon-after re-read in a repeating cycle due to a mismatch of demand vs. capacity.

Typically the OS kernel's filesystem cache is larger than the postgres shared-buffer pool, so when postgres evicts a page from a shared buffer and then quickly re-reads it back into another shared buffer, often that does not require physical disk IO. Rather, the eviction overhead is the main concern, because it drives serialization.

Solutions to this contention include:

- Reduce query concurrency per postgres instance:
 - This can take the form of:
 - * Adding another replica db node to spread the workload across.
 - * Reducing pgbouncer backend connection pool sizes
 - Rationale: Fewer concurrently active backends should reduce the eviction rate (and the contention it drives) by reducing the working set size to better fit in the shared buffer pool. The shared buffer pool is a per-postgres-instance resource, so this tactic aims to reduce per-instance concurrency (either by reducing concurrent client connections or adding nodes for extra capacity).
 - * How effective this is depends on the severity of the contention.
 - * If contention affects the replicas, adding more replicas should always help.
 - * Moderately reducing pgbouncer backend pool size may help and may be worth trying if contention affects the primary (where adding nodes is not an option).
- Query tuning to reduce buffer usage:
 - If any frequently run queries access many buffers (especially if the shared buffer pool spends a large proportion of its pages on the tables+indexes used by frequently run queries), check if those queries could be adjusted to use fewer buffers (e.g. use a better index, vacuum more often to clean up dead index item pointers).
 - Or can that query be run less often (e.g. results-caching or special-casing)? Or refactored to make the common case cheaper?
- Grow the **shared_buffers** pool size:
 - Rarely a first choice, for several reasons:
 - * Requires a postgres restart (intrusive). That restart temporarily

- reduces fleet capacity (down by 1 replica), driving more traffic onto the (already struggling) remaining replica nodes.
- * Overhead trade-offs may potentially make things worse. A larger pool of shared buffers should have a lower eviction rate at the same query concurrency (because the working set fits better into the pool), but its per-eviction overhead increases (which may offset the benefit of a lower eviction rate).
 - * Lastly, growing **shared_buffers** implicitly reduces the memory available for other purposes (e.g. filesystem cache, **work_mem**, etc.). A proportionally large change to **shared_buffers** could introduce unwanted side-effects, such as shifting the bottleneck (e.g. to disk IO latency) without actually improving the query throughput and response times.
- If we decide to try increasing **shared_buffers**, aim for a controlled experimental setup that is quick and safe to revert.
 - * Example: Provision a new replica with a larger **shared_buffers** and otherwise identical specs and config. Allow for cache warming when first introduced to production traffic. Once stable state is reached, consider setting **patroni**'s **noloadbalance** tag on one of the normal replicas; it remains a viable up-to-date replica, but clients will ignore it until we remove that tag. This reduces the count of active replicas to what it initially was, but now one of them has the adjusted **shared_buffers** setting. Since all active replicas should have comparable query rates, we can directly compare their usage metrics and concurrently capture any desired profiling runs. At any point, we can quickly and safely end the experiment by adding the **noloadbalance** tag to the experimental node and removing that tag from the normal healthy node.